

**Escola Superior de Tecnologia e Gestão
Licenciatura em Engenharia Informática**

Estrutura de Dados 2025/26

**Trabalho prático nº 2
Tipos de Serviço**

Bernardo José Inácio Gomes 2025145298

Elaborado em: 20/06/2026

Índice

Lista de Figuras.....	ii
1. Introdução	1
2. Organização da aplicação	2
2.1. Struct Tipo de Serviços(estruturaServicos)	2
2.2. Struct Serviços Individuais(estruturaServicoIndividual).....	2
3. Fluxo de Vida	3
3.1. Adicionar Tipos de Serviço	3
3.2. Remover um tipo de serviço	3
3.3. Adicionar um serviço individual	4
3.4. Remover um serviço individual	4
3.5. Editar um tipo de Serviço	5
3.6. Editar números de componentes e valores	5
4. Integridade da aplicação.....	6
4.1. Serviços cancelados e adicionar novos serviços individuais	6
4.2. Conclusão de um serviço e comparação com data de início e de fim	7
5. Cálculos de valores a pagar, médias e duração de serviços	8
5.1. Cálculo de um valor a pagar	8
5.2. Média dos preços dos serviços concluídos	10
5.3. Durações de serviços concluídos (maior, menor e média)	12
6. Persistência de dados	13
6.1. Guardar dados (Escrita de dados)	13
6.2. Escrever dados (Leitura de Dados).....	14
7. Algoritmos	15
7.1. Bubble Sort.....	15
7.2. Selection Sort	16
8. Recursividade.....	17
9. Testes e análise	18
9.1. Teste de persistência de dados	18
9.2. Ordenação por ordem decrescente dos serviços concluídos por valor a pagar	19
9.3. Calcular a duração mínima, a duração máxima e a duração média de todos os serviços concluídos	20
10. Conclusão	21
11. Referências	23

Lista de Figuras

FIGURA 1 – FUNÇÃO PARA JUNTAR NOVA LISTA AO INÍCIO.....	6
FIGURA 2 – FUNÇÃO PARA CANCELAR UM TIPO DE SERVIÇO	6
FIGURA 3 – COMPARAÇÃO ENTRE AS DATAS DE FIM E INÍCIO	7
FIGURA 4 – SUBPROGRAMA PARA CALCULAR O VALOR A PAGAR.....	8
FIGURA 5 – SUBPROGRAMA PARA CALCULAR O VALOR A PAGAR.....	9
FIGURA 6 – SUBPROGRAMA PARA NÚMERO DE SERVIÇOS CONCLUÍDOS.....	10
FIGURA 7 – SUBPROGRAMA PARA O CÁLCULO DO PREÇO DE UM SERVIÇO CONCLUÍDO	10
FIGURA 8 – FUNÇÃO PARA CALCULAR A MEDIA DE PREÇOS DE SERVIÇOS CONCLUÍDOS	11
FIGURA 9 – CÁLCULO DE DURAÇÃO DE SERVIÇOS E MÉDIAS DE DURAÇÃO.....	12
FIGURA 10 – SUBPROGRAMA PARA GUARDAR DADOS.....	13
FIGURA 11 – SUBPROGRAMA PARA CARREGAR DADOS.....	14
FIGURA 12 – SUBPROGRAMA PARA ORDENAR DE FORMA DECRESCENTE OS SERVIÇOS CONCLUÍDOS ATRAVÉS DE VALOR PAGO	15
FIGURA 13 – SUBPROGRAMA PARA ORDENAR DE FORMA DECRESCENTE OS SERVIÇOS POR TIPO ATRAVÉS DO NÚMERO DE SERVIÇOS A DECORRER.....	16
FIGURA 14 - FUNÇÃO RECURSIVA.....	17
FIGURA 15 - FUNÇÃO DE CONTAGEM DO NÚMERO DE SERVIÇOS A DECORRER.....	17
FIGURA 16 - PERSISTÊNCIA DE DADOS.....	18
FIGURA 17 – DADOS DOS CONCLUÍDOS ANTES DA ORDENAÇÃO.....	19
FIGURA 18 - ORDENAÇÃO DOS DADOS APÓS CHAMADA DA FUNÇÃO DE ORDENAÇÃO.....	19
FIGURA 19 - RESULTADO DO CÁLCULO DAS DURAÇÕES DE TODOS OS SERVIÇOS CONCLUÍDOS	20
FIGURA 20 - NOVA FUNÇÃO DE CARREGAR DADOS	21
FIGURA 21 - NOVA FUNÇÃO DE CARREGAR DADOS	22

1. Introdução

Este trabalho tem como objetivo registar e organizar os serviços fornecidos por uma loja de reparações informáticas.

Para a organização e gestão dos tipos de serviço foi utilizada uma tabela em que cada tipo de serviço tem dois ponteiros, um para serviços em execução e outro para serviços concluídos.

A realização deste trabalho tem como base conteúdos lecionados na disciplina de Estrutura de Dados tais como: listas ligadas, ponteiros, algoritmos de ordenação.

Tendo como base alguns aspetos do Trabalho nº1 as datas são colocadas da mesma maneira ou seja:
Dia Mes Hora Minuto Segundo Ano.

Por exemplo para escrever a data 17/04/2026 às 09:46, no programa será da seguinte forma:
17 04 09 46 00 2026.

2. Organização da aplicação

Este capítulo refere-se ao modo como a aplicação está organizada, ou seja, qual a base pela qual a aplicação irá funcionar.

Tendo como base o trabalho anterior (Trabalho Prático nº 1), foi criada uma **struct** referente ao **tipo de serviço(estruturaServicos)** e outra para os **serviços individuais(estruturaServicoIndividual)**.

2.1. Struct Tipo de Serviços(estruturaServicos)

Esta struct tipo de serviços contém menos campos do que o trabalho anterior sendo eles:

- Nome do Funcionário (*char*)
- Tipo de Serviço (*char*)
- Preço do Serviço(*float*)
- Variáveis de estado do tipo inteiro (cancelado, pago)
- Dois ponteiros do mesmo tipo (pEstrutura) um para os serviços em execução e outro para os serviços concluídos.

Esta struct serve para poder armazenar numa tabela os tipos de serviços, onde posteriormente a cada tipo de serviço serão acrescentados os serviços individuais.

A tabela neste trabalho tem um tamanho muito menor do que no trabalho anterior já que esta se refere aos tipos de serviços.

Para este trabalho foram escolhidos quatro tipos de serviços (Manutenção, Diagnóstico, Reparação, Limpeza).

2.2. Struct Serviços Individuais(estruturaServicoIndividual)

Esta estrutura permite obter os dados para os serviços individuais onde cada serviço irá ter:

- Nome do Cliente(**char**)
- Equipamentos(**char**)
- Observações(**char**)
- Código de serviço Id(**int**)
- Número de componentes(**int**)
- Uma **struct** do tipo Data (Data de Inicio, Data de Fim), utilizada do trabalho anterior.
- Um **array** do tipo **float** para guardar os preços dos componentes substituídos.
- Variáveis de estado do tipo inteiro (pago, concluido).
- Um ponteiro do tipo (pEstrutura) para apontar para o próximo serviço

3. Fluxo de Vida

Este capítulo refere-se ao funcionamento da aplicação, serão dados alguns exemplos de como a aplicação funciona.

3.1. Adicionar Tipos de Serviço

Para adicionar um tipo de Serviço o utilizador escolhe a opção do menu “Registar tipo de serviço”, caso o número total de serviços seja menor que o tamanho da tabela então é chamado o subprograma de registo.

Caso esse tipo de serviço já exista a aplicação não permite adicionar um novo tipo de serviço.

O subprograma de registo coloca as variáveis de estado a zero e os ponteiros de execução e concluídos a NULL, o utilizador seleciona assim o nome do tipo de serviço e onde consoante o tipo de serviço selecionado existe um preço por aquele tipo.

No caso de o serviço ser diagnóstico ou limpeza o preço é fixo, mas para o caso de manutenção e reparação o preço de serviço é cobrado por hora.

Após a seleção do tipo de serviço o estado de registado fica com o valor de um indicando assim que aquele tipo de serviço já foi registado.

Este subprograma devolve uma estrutura do tipo Servico.

3.2. Remover um tipo de serviço

O utilizador também pode remover um tipo de serviço, através da seleção no menu da opção “Remover Tipo de Servico”.

Nesta opção é chamado o subprograma eliminarTipoServico (*totalServicos, servicos), onde são passados por parâmetros o endereço do tamanho da tabela e a tabela.

O utilizador digita o tipo de serviço que deseja remover, a função ao encontrar o tipo de serviço chama um outro subprograma para remover as listas de execução e concluídos e finalmente é decrementado o número total de tipos de serviços.

3.3. Adicionar um serviço individual

Para adicionar um serviço individual o utilizador seleciona a opção no menu “Adicionar um serviço”, esta opção chama o subprograma `juntarInicio` (`totalServicos`, `servicos`) , cujo parâmetros são o tamanho da tabela e a tabela.

O objetivo deste subprograma é através do ponteiro de execução de um tipo de serviço, adicionar um serviço individual.

O utilizador escolhe o tipo de serviço (Reparação, Manutenção, Diagnóstico, Limpeza), este serviço tem de estar registado e não pode estar cancelado.

Após as verificações o utilizador coloca o nome do cliente, nome do equipamento, o código do serviço, e por fim a data de início do serviço.

Através da função ***malloc*** é reservado um espaço na memória com o tamanho de um serviço individual e devolve o ponteiro(`no`) para aquele espaço.

Os valores digitados pelo utilizador são preenchidos na estrutura de serviço individual através do ponteiro devolvido pela função ***malloc***.

Por fim este ponteiro é direcionado para o início da lista através da utilização do ponteiro de execução.

3.4. Remover um serviço individual

Para remover um serviço individual o utilizador seleciona no menu de opções a opção “Eliminar um Serviço”, esta opção irá chamar o subprograma `eliminarServico` (`totalServicos`, `servicos`), cujo parâmetros são o tamanho da tabela e a tabela.

O utilizador coloca o tipo de serviço que deseja eliminar podendo estes serem (Manutenção, Reparação, Diagnóstico ou Limpeza).

O tipo de serviço que o utilizador colocou não pode estar cancelado, ao encontrar o tipo de serviço o utilizador terá de colocar o código do serviço individual que foi definido pelo mesmo.

Através da utilização de dois ponteiros e com o uso da função ***free*** é possível remover o serviço indicado pelo código.

3.5. Editar um tipo de Serviço

Também é possível editar um tipo de serviço, ou seja, o utilizador consegue editar dois parâmetros sendo estes o nome do funcionário e o preço do serviço.

Basta ao utilizador seleccionar a opção no menu de “Editar tipo de serviço”, e colocar o nome do tipo de serviço que deseja editar.

3.6. Editar números de componentes e valores

Esta funcionalidade permite editar o número de componentes que foram substituídas e os respetivos preços.

Esta função só permite ao utilizador editar nos tipos de serviço de reparação e manutenção.

O utilizador digita o tipo de serviço, caso este não seja de reparação ou manutenção a função não permite a edição caso contrário o utilizador terá de digitar o código de serviço pretendido.

Através do uso de um ponteiro auxiliar, a função permite saber se aquele serviço individual já está pago ou concluído e caso estas flags sejam verdadeiras a função não permitirá a edição.

Caso contrário o utilizador depois colocará o nome do componente substituído e o seu valor, através do ponteiro auxiliar é atualizado a tabela dos preços dos componentes e o número de componentes substituídos.

4. Integridade da aplicação

Este capítulo visa aspetos relacionados com maneira como alguns subprogramas geram os dados mediante determinadas condições ou comparações.

4.1. Serviços cancelados e adicionar novos serviços individuais

Como referido anteriormente para adicionar um novo serviço individual este só poderá ser adicionado caso o tipo de serviço não esteja cancelado e o tipo de serviço esteja registado.

```
void juntarInicio(int totalServicos , Servico servicos[]){  
  
    char nomeClienteAux[100] , equipamentoAux[100] , tipoServicoAux[100];  
    int codigoAux , diaAux , mesAux , horaAux , minutoAux , segundoAux , anoAux;  
    int flagRegisto = 0;  
  
    if(totalServicos > 0){  
        //Se for um pedir os dados.  
        printf("Digite o tipo de servico: \n");  
        fgets(tipoServicoAux , 100 , stdin);  
        removerEnter(tipoServicoAux);  
  
        for(int i = 0 ; i < totalServicos ; i++){  
  
            if(servicos[i].registado == 1 && servicos[i].cancelado == 0 && strcasecmp(servicos[i].  
            tipoServico , tipoServicoAux) == 0){  
  
                printf("Digite o nome do cliente: ");  
                fgets(nomeClienteAux , 100 , stdin);  
                removerEnter(nomeClienteAux);  
  
                printf("Digite o nome do equipamento: ");  
                fgets(equipamentoAux , 100 , stdin);  
                removerEnter(equipamentoAux);  
            }  
        }  
    }  
}
```

Figura 1 – Função para juntar nova lista ao início

A condição dentro do loop indica mesmo isso, ou seja, caso o serviço esteja registado(servicos[i].registado) e não esteja cancelado(servicos[i].cancelado) o utilizador poderá preencher os dados daquele serviço individual.

```
//Cancelar Tipo de Serviço  
void cancelarTipoServico(int totalServicos , Servico servicos[]){  
    char serv[100];  
    printf("Digite o tipo de servico que deseja cancelar: ");  
    fgets(serv , 100 , stdin);  
    removerEnter(serv);  
  
    for(int i = 0 ; i < totalServicos ; i++){  
        if(strcasecmp(servicos[i].tipoServico , serv) == 0){  
            servicos[i].cancelado = 1;  
        }  
    }  
}
```

Figura 2 – Função para cancelar um tipo de serviço

Esta é o subprograma que permite cancelar um tipo de serviço.

4.2. Conclusão de um serviço e comparação com data de início e de fim

Para concluir um serviço existe uma comparação importante que tem de ser feita sendo esta a seguinte, a data de fim ou de conclusão não pode ser menor do que a data de início ou a data de registo daquele serviço.

```
printf("Insira a data de fim no formato de inteiros: (Dia Mes hh mm ss 2026) \n");
scanf("%d %d %d %d %d %d", &atual -> dataDefim.dia, &atual -> dataDefim.mes, &atual -> dataDefim.hora, &atual -> dataDefim.minuto, &atual -> dataDefim.segundo, &atual -> dataDefim.ano);
getchar();

time_t ti = construirEstrutura(atual -> dataDeInicio.hora, atual -> dataDeInicio.minuto, atual -> dataDeInicio.segundo, atual -> dataDeInicio.dia, atual -> dataDeInicio.mes, atual -> dataDeInicio.ano);
time_t tf = construirEstrutura(atual -> dataDefim.hora, atual -> dataDefim.minuto, atual -> dataDefim.segundo, atual -> dataDefim.dia, atual -> dataDefim.mes, atual -> dataDefim.ano);

diferenca = difftime(tf, ti);

if(diferenca > 0){
    atual -> concluido = 1;
    if(anterior == NULL){
        servicos[indice].execucao = atual -> prox;
    }
    else{
        anterior -> prox = atual -> prox;
    }
}

printf("Data de Conclusao: %02d/%02d/%04d %02d:%02d:%02d ",
    atual->dataDefim.dia,
    atual->dataDefim.mes,
    atual->dataDefim.ano,
    atual->dataDefim.hora,
    atual->dataDefim.minuto,
    atual->dataDefim.segundo);

atual -> prox = servicos[indice].concluidos;
servicos[indice].concluidos = atual;
atual -> concluido = 1;
printf("Serviço concluído com sucesso!\n");
```

Figura 3 – Comparação entre as datas de fim e início

Esta imagem é referente ao subprograma de concluir um serviço individual, através da utilização de um subprograma anterior de **construirEstrutura** onde este devolve um tipo de dado especial que é o **time_t** (**long long**) é possível calcular a diferença de tempo das datas de início e de fim.

As datas de inicio e de fim são acedidas através de um ponteiro (**atual**), e passadas como parâmetros para **construirEstrutura**, a variável diferença armazena a diferença de tempo entre **tf** e **ti** que é calculada pela função **difftime**.

Caso a diferença seja maior que zero o que significa que a data de fim é maior do que a data de início, os ponteiros de atual e anterior são ajustados e os dados respetivos das datas são preenchidos na estrutura selecionada pelo respetivo código, através do ponteiro atual.

5. Cálculos de valores a pagar, médias e duração de serviços

Este capítulo refere-se à secção do programa voltada para cálculos e alguns dados relativamente a médias e serviços com maior ou menor duração.

5.1. Cálculo de um valor a pagar

```
//Calcular o valor a pagar
float valorPagar(int totalServicos , Servico servicos[]){
    char tipoServicoAux[100] , nomeComp[100];
    int encontrou = 0 , indice , codigoAux , editavel = 0;
    float valor;
    float precoServicoAux ;
    pEstrutura atual = NULL;
    pEstrutura anterior = NULL;
    printf("Digite o tipo de servico que deseja procurar: ");
    fgets(tipoServicoAux , 100 , stdin);
    removerEnter(tipoServicoAux);

    for(int i = 0 ; i < totalServicos ; i++){
        if(strcasecmp(servicos[i].tipoServico , tipoServicoAux) == 0){
            atual = servicos[i].execucao;
            encontrou = 1;
            indice = i;
        }
    }

    if(encontrou){
        printf("Digite o numero do servico para obter o valor a pagar: \n");
        scanf("%d" , &codigoAux);
        getchar();

        atual = servicos[indice].execucao;
        while(atual != NULL && atual -> codigoServicoID != codigoAux){
            atual = atual -> prox;
        }

        if(atual == NULL){
            atual = servicos[indice].concluidos;

            while(atual != NULL && atual -> codigoServicoID != codigoAux){
                atual = atual -> prox;
            }

            if(atual == NULL){
                printf("Nao foi possivel encontrar o servico!\n");
                return 0.0;
            }
        }

        float somaComponentes = 0.0;
        for(int i = 0 ; i < atual -> numComponentes ; i++){
            somaComponentes = somaComponentes + atual -> precoComponentes[i];
        }

        if(strcasecmp(servicos[indice].tipoServico , "Reparacao") == 0 || strcmp(servicos[indice].tipoServico , "Manutencao") == 0){
            time_t t1 = construirEstrutura(atual -> dataDeInicio.hora , atual -> dataDeInicio.minuto , atual -> dataDeInicio.segundo , atual -> dataDeInicio.dia , atual -> dataDeInicio.mes , atual -> dataDeInicio.ano);
            time_t tf;
        }
    }
}
```

Figura 4 – Subprograma para calcular o valor a pagar

A imagem acima refere-se ao cálculo do valor a pagar por um serviço individual, foi utilizado um ponteiro auxiliar para percorrer tanto os serviços em execução como os concluídos caso este tivesse o valor NULL em ambos então o serviço não estaria disponível.

Caso contrário o ponteiro atual poderia estar ou nos serviços em execução ou nos serviços concluídos, através do ponteiro atual foi possível aceder aos valores dos preços dos componentes e efectuar a soma soma.

No caso de o tipo de serviço ser reparação ou manutenção obteve-se o valor em formato **long** **long(time_t)** da data de início.

```
36         if(atual -> concluido == 1){
37             tf = construirEstrutura(atual -> dataDefim.hora , atual -> dataDefim.minuto , atual -> dataDefim.segundo , atual -> dataDefim.dia , atual -> dataDefim.
38                 mes , atual -> dataDefim.ano);
39         }
40     }
41
42     else{
43         tf = time(NULL);
44     }
45
46     double segundos = difftime(tf , ti);
47     if(segundos < 0){
48         segundos = 0;
49     }
50     precoServicoAux = (((float)segundos / 3600.0) * servicos[indice].precoServico) + somaComponentes;
51 }
52
53 else{
54     precoServicoAux = servicos[indice].precoServico;
55 }
56
57 }
58
59 else{
60     printf("Servico nao encontrado !\n");
61 }
62
63 return precoServicoAux;
64 }
65
66 }
67 }
```

Figura 5 – Subprograma para calcular o valor a pagar

Se o serviço estivesse ainda em execução este não teria um tempo final ou de conclusão então adotou-se o seguinte ***tf = time (NULL);***

O que para este caso significa contar o tempo desde a data de início até ao tempo atual.

Para finalizar o cálculo do tempo efetuou-se a diferença entre ***ti*** e ***tf***, assim acedendo ao preço do tipo de serviço através do índice, multiplicando este pelo tempo e somando o conjunto ao número de componentes obtém assim o valor a pagar para o caso do serviço ser de manutenção ou reparação e caso este já esteja concluído ou não.

Para os restantes serviços como diagnóstico e limpeza o preço é sempre fixo pelo que a variável **preçoServicoAux** é igual a **servicos[indice].precoServico**.

5.2. Média dos preços dos serviços concluídos

Para implementar esta funcionalidade fez-se o uso de três subprogramas.

```
//Numero de Servicos Concluidos

int totalServicosConcluidos(int totalServicos , Servico servicos[]){
    int contador = 0;
    for(int i = 0 ; i < totalServicos ; i++){
        pEstrutura atual = servicos[i].concluidos;

        while(atual != NULL){
            contador++;
            atual = atual -> prox;
        }
    }
    return contador;
}
```

Figura 6 – Subprograma para número de serviços concluídos

Este subprograma utiliza um ponteiro auxiliar para efectuar a contagem do número de serviços concluídos através de um contador.

Retorna assim um valor do tipo inteiro com o número total de serviços individuais concluídos.

```
07 //Calcular Preço de um servico
08 float precoServicoConcluido(Servico tipo , pEstrutura atual){
09     float somaPreco = 0;
10     for(int i = 0 ; i < atual -> numComponentes ; i++){
11         somaPreco += atual -> precoComponentes[i];
12     }
13
14     if(strcasecmp(tipo.tipoServico , "Reparacao") == 0 || strcmp(tipo.tipoServico , "Manutencao") == 0){
15
16         time_t ti = construirEstrutura(atual -> dataDeInicio.hora , atual -> dataDeInicio.minuto , atual -> dataDeInicio.segundo , atual -> dataDeInicio.dia , atual
17         dataDeInicio.mes , atual -> dataDeInicio.ano);
18
19         time_t tf = construirEstrutura(atual -> dataDeFim.hora , atual -> dataDeFim.minuto , atual -> dataDeFim.segundo , atual -> dataDeFim.dia , atual -> dataDeFim
20         mes , atual -> dataDeFim.ano);
21
22         double segundos = difftime(tf , ti);
23
24         if(segundos < 0 ){
25             segundos = 0;
26         }
27         somaPreco += (segundos / 3600) * tipo.precoServico;
28     }
29     else{
30         somaPreco += tipo.precoServico;
31     }
32     return somaPreco;
33 }
```

Figura 7 – Subprograma para o cálculo do preço de um serviço concluído

Este subprograma tem como finalidade efetuar o cálculo de um determinado tipo de serviço, utilizando por parâmetros o tipo de serviço e um ponteiro, retornando um valor do tipo float.

O ponteiro serve para obter a soma de cada valor de cada componente, e posteriormente para aceder às datas de início e de fim.

Ou seja, consoante o tipo de serviço passado o subprograma calcula com base na duração do serviço o preço deste.

Caso o tipo de serviço não seja nem de reparação nem de manutenção o preço do serviço é fixo ou seja não é necessário o cálculo do tempo.

```
//Calcula a media dos precos dos servicos concluidos
float calcularMedia_precosServicos_Concluidos(int totalServicos , Servico servicos[] , int servicosConcluidos){
    float precoTotal = 0 ;
    for(int i = 0 ; i < totalServicos ; i++){
        pEstrutura atual = servicos[i].concluidos;

        while(atual != NULL){
            precoTotal += precoServicoConcluido(servicos[i] , atual);
            atual = atual -> prox;
        }
    }

    if(servicosConcluidos == 0){
        return 0;
    }
    else{
        return precoTotal / servicosConcluidos;
    }
}
```

Figura 8 – Função para calcular a media de preços de serviços concluídos

Por fim, o subprograma acima efetua os cálculos finais chamando assim o subprograma anterior, o parâmetro `servicosConcluidos` é passado no main através da chamada do subprograma `totalServicosConcluidos`.

Através do uso do ponteiro `atual` este aponta inicialmente para os concluídos de cada tipo de serviço, onde depois cada tipo de serviço e o ponteiro respetivo é passado para o subprograma `precoServicoConcluido` onde cada preço é somado e no fim é calculada a média e devolvido um valor do tipo `float`.

5.3. Durações de serviços concluídos (maior, menor e média)

Este subprograma tem como objetivo o cálculo das durações mínimas, máximas e a duração média de todos os serviços concluídos.

```
//Duração média, máxima e mínima de todos os serviços
void duracaoDeTodosServicos(int totalServicos, Servico servicos[]){
    double maior = 0;
    double menor = 999999999;
    int totalServicosConcluidosAux = totalServicosConcluidos(totalServicos, servicos);
    double media, contador = 0;
    pEstrutura pAtual = NULL;
    pEstrutura pAuxMaior = NULL;
    pEstrutura pAuxMenor = NULL;

    for(int i = 0; i < totalServicos; i++){
        pAtual = servicos[i].concluidos;

        while(pAtual != NULL){
            time_t ti = construirEstrutura(pAtual -> dataDeInicio.hora, pAtual -> dataDeInicio.minuto, pAtual -> dataDeInicio.segundo, pAtual -> dataDeInicio.dia, pAtual -> dataDeInicio.mes, pAtual -> dataDeInicio.ano);
            time_t tf = construirEstrutura(pAtual -> dataDeFim.hora, pAtual -> dataDeFim.minuto, pAtual -> dataDeFim.segundo, pAtual -> dataDeFim.dia, pAtual -> dataDeFim.mes, pAtual -> dataDeFim.ano);

            double tempoFinal = difftime(tf, ti);
            contador = contador + tempoFinal;

            if(tempoFinal > maior){
                maior = tempoFinal;
                pAuxMaior = pAtual;
            }

            if(tempoFinal < menor){
                menor = tempoFinal;
                pAuxMenor = pAtual;
            }

            pAtual = pAtual -> prox;
        }

        if(totalServicosConcluidosAux == 0){
            printf("Nao existem servicos concluidos");
            return;
        }

        else{
            media = contador / (double)(totalServicosConcluidosAux);

            printf("Maior duracao: \n");
            printf("Codigo Servico: %s\n", pAuxMaior -> codigoServicoID);
            printf("Nome Cliente: %s\n", pAuxMaior -> cliente);
            printf("Equipamento: %s\n", pAuxMaior -> equipamento);
            printf("Valor: %.2f\n", maior);

            printf("Menor Duracao: \n");
            printf("Codigo Servico: %s\n", pAuxMenor -> codigoServicoID);
            printf("Nome Cliente: %s\n", pAuxMenor -> cliente);
            printf("Equipamento: %s\n", pAuxMenor -> equipamento);
            printf("Valor: %.2f\n", menor);

            printf("Media de duracao: %.2f\n", media);
        }
    }
}
```

Figura 9 – Cálculo de duração de serviços e médias de duração.

Através da utilização do ponteiro pAtual foi possível aceder ao ponteiro dos concluídos para cada tipo de serviço.

Após essa implementação o ponteiro pAtual foi utilizado para percorrer cada serviço individual que estivesse concluído obtendo assim o valor em **time_t** para as datas de início e de fim, foi feito o cálculo das diferenças de tempo e armazenando em cada variável o maior tempo e o menor tempo.

A utilização dos ponteiros pMaior e pMenor teve como objetivo a visualização dos valores mínimos e máximos de durações.

Para o cálculo da média foi utilizado o subprograma anterior cuja a finalidade é obter o total de serviços concluídos.

6. Persistência de dados

Este capítulo é referente à leitura e escrita de dados por parte da aplicação através da utilização de ficheiros binários. Para guardar os dados no fim de cada operação dentro da aplicação basta carregar no zero e o programa termina e guarda as listas no ficheiro.

6.1. Guardar dados (Escrita de dados)

```
void guardarDados(int totalServicos, Servico servicos[]){  
    FILE *f = fopen("dados.dat", "wb");  
  
    if(f == NULL){  
        printf("Erro ao abrir o ficheiro!");  
        return;  
    }  
  
    fwrite(&totalServicos, sizeof(int), 1, f);  
  
    for(int i = 0; i < totalServicos; i++){  
        fwrite(&servicos[i], sizeof(Servico), 1, f);  
  
        pEstrutura atual = servicos[i].execucao;  
        while(atual != NULL){  
            fwrite(atual, sizeof(servicoIndividual), 1, f);  
            atual = atual->prox;  
        }  
  
        servicoIndividual marcadorFim = {marcadorFim.codigoServicoID - 1};  
        fwrite(&marcadorFim, sizeof(servicoIndividual), 1, f);  
  
        atual = servicos[i].concluidos;  
        while(atual != NULL){  
            fwrite(atual, sizeof(servicoIndividual), 1, f);  
            atual = atual->prox;  
        }  
  
        fwrite(&marcadorFim, sizeof(servicoIndividual), 1, f);  
    }  
  
    fclose(f);  
    printf("Servicos Guardados!\n");  
}
```

Figura 10 – Subprograma para guardar dados

Este subprograma consiste no armazenamento das listas para um ficheiro binário, onde primeiramente é armazenado para o total de serviços o número total dos serviços atual.

Depois é percorrida a tabela com os tipos de serviço, onde para cada tipo de serviço as listas de execução e concluídas são escritas no ficheiro ("data.dat").

É utilizado um marcador final com o código de serviço -1 para sinalizar o fim da escrita para cada uma das listas.

6.2. Escrever dados (Leitura de Dados)

```
//Carregar Dados
int carregarDados(Servico servicos[]){
    FILE *f = fopen("dados.dat", "rb");

    if(f == NULL){
        printf("Nao foi possivel abrir o ficheiro\n");
        return 0;
    }
    int totalServicos = 0;

    fread(&totalServicos, sizeof(int), 1, f);

    for(int i = 0; i < totalServicos; i++){
        fread(&servicos[i], sizeof(Servico), 1, f);

        servicos[i].execucao = NULL;
        servicos[i].concluidos = NULL;

        servicoIndividual aux;

        fread(&aux, sizeof(servicoIndividual), 1, f);

        while(aux.codigoServicoID != -1){

            pEstrutura novo = (pEstrutura)malloc(sizeof(servicoIndividual));

            if(novo != NULL){
                *novo = aux;
                novo -> prox = servicos[i].execucao;
                servicos[i].execucao = novo;
            }

            fread(&aux, sizeof(servicoIndividual), 1, f);
        }

        fread(&aux, sizeof(servicoIndividual), 1, f);

        while(aux.codigoServicoID != -1){

            pEstrutura novoConcluidos = (pEstrutura)malloc(sizeof(servicoIndividual));

            if(novoConcluidos != NULL){
                *novoConcluidos = aux;
                novoConcluidos -> prox = servicos[i].concluidos;
                servicos[i].concluidos = novoConcluidos;
            }

            fread(&aux, sizeof(servicoIndividual), 1, f);
        }
    }

    fclose(f);
    printf("Servicos carregados com sucesso\n");
    return totalServicos;
}
```

Figura 11 – Subprograma para carregar dados

Na leitura dos dados a partir do ficheiro binário, é utilizado uma estrutura auxiliar que permite fazer um ciclo enquanto os ponteiros não chegarem ao marcador menos um.

É criado um ponteiro novo que aponta para um espaço da memória onde ficará a apontar para o início da lista de execução, para as listas de concluídos é utilizado o ponteiro novoConcluidos.

7. Algoritmos

Este capítulo irá se debruçar sobre dois algoritmos de ordenação utilizados (Bubble Sort, Selection Sort)

7.1. Bubble Sort

```
//Ordenar Servicos Concluidos Por Ordem decrescente do valor pago
void ordenarServicosConcluidos(Servico *tipo){
    if(tipo -> concluidos == NULL){
        printf("Nao e possivel encontrar o servico\n");
        return;
    }

    pEstrutura i, j;

    for(i = tipo->concluidos ; i != NULL ; i = i -> prox){
        for(j = i -> prox ; j != NULL ; j = j -> prox){
            float valor_i = precoServicoConcluido(*tipo, i);
            float valor_j = precoServicoConcluido(*tipo, j);

            if(valor_i < valor_j){
                servicoIndividual tmp = *i;
                *i = *j;
                *j = tmp;

                pEstrutura aux = i ->prox;
                i -> prox = j ->prox;
                j ->prox = aux;
            }
        }
    }
}
```

Figura 12 – Subprograma para ordenar de forma decrescente os serviços concluídos através de valor pago

Este algoritmo foi utilizado para ordenar os serviços concluídos por ordem decrescente do valor pago aqui é chamado um outro subprograma que é o `precoServicoConcluido` que retorna o valor do preço de cada serviço concluído.

Armazenando esse valor nas variáveis `valor_i` e `valor_j` é depois possível comparar os valores pagos de um elemento da lista e o seu seguinte.

Através da utilização dos ponteiros `i` e `j` e de uma estrutura auxiliar do mesmo tipo, a posição na lista de um determinado elemento é trocada caso o valor de `i` seja menor que o valor de `j`.

7.2. Selection Sort

```
//Ordenar os tipos de serviços por ordem decrescente do número de serviços a decorrer. (Selection Sort)
void ordenarDecrescenteTipos(int totalServicos , Servico servicos[]) {

    int indiceMaximo;
    Servico tmp;
    for(int i = 0 ; i < totalServicos-1 ; i++){
        indiceMaximo = i;
        for(int j = i + 1 ; j < totalServicos ; j++){
            if(numeroServicosDecorrer(servicos[j]) > numeroServicosDecorrer(servicos[i])){
                indiceMaximo = j;
            }
        }

        if(indiceMaximo != i){
            tmp = servicos[i];
            servicos[i] = servicos[indiceMaximo];
            servicos[indiceMaximo] = tmp;
        }
    }
}
```

Figura 13 – Subprograma para ordenar de forma decrescente os serviços por tipo através do número de serviços a decorrer

Este subprograma pretende ordenar por ordem decrescente os tipos de serviço com base no número de serviços a decorrer.

É utilizada uma variável de índice máximo, com o objetivo de perceber qual é o índice máximo e se este foi alterado dentro do loop para j.

Caso este índice máximo seja alterado a condição `indiceMaximo != i`, irá permitir a troca dos tipos de serviço, também o número de serviços a decorrer foi obtido através da chamada do subprograma `numeroServicosDecorrer`.

8. Recursividade

Para resolver o requisito de recursividade recorreu-se a um subprograma simples para contar o número de nós, este subprograma foi chamado no subprograma para contar o número de nós nos serviços a decorrer, contar o número de serviços cancelados.

```
//Funcao Recursiva para contar nós
int contarNos(pEstrutura atual){
    if(atual == NULL){
        return 0;
    }

    return 1 + contarNos(atual -> prox);
}
```

Figura 14 - Função Recursiva

```
//Obter numero de servicos a decorrer
int numeroServicosDecorrer(Servico servico){
    pEstrutura atual = servico.execucao;
    int contadorServico = 0;

    contadorServico += contarNos(atual);

    return contadorServico;
}
```

Figura 15 - Função de Contagem do Número de Serviços a decorrer

Através da passagem do ponteiro referente ao serviço em execução para a função recursiva de contar nós é possível obter o número de serviços a decorrer, este subprograma que devolve o número de serviços a decorrer é utilizado na ordenação decrescente dos tipos de serviço.

9. Testes e análise

9.1. Teste de persistência de dados

Este teste tem como objetivo verificar o funcionamento da escrita e leitura dos dados.

```
Funcionario: Bernardo
Preco do Servico: 5.00
[Codigo Servico ID: 1 Nome do Cliente: Artur Observacoes: Pago: 0]
A lista esta vazia!
Funcionario: Maria
Preco do Servico: 20.00
[Codigo Servico ID: 2 Nome do Cliente: Ramiro Observacoes: Pago: 0]
A lista esta vazia!
  2 - Registrar Tipo de Servico
  3 - Apresentar tabela
  4 - Editar Dados de um tipo de servico
```

Figura 16 - Persistência de Dados

Para este teste foram realizados os seguintes passos:

1. Registrar um tipo de serviço (Opção 2).
2. Adicionar um serviço (Opção 7).
3. Registrar um segundo tipo de serviço (Opção 2).
4. Adicionar um serviço para este tipo de serviço (Opção 7).
5. Sair da aplicação (Opção 0).

Na imagem poderá se ver a mensagem de “A lista esta vazia!” esta mensagem é referente aos serviços concluidos.

9.2. Ordenação por ordem decrescente dos serviços concluídos por valor a pagar

Este teste tem como objetivo fazer a ordenação por ordem decrescente do valor a pagar, dos serviços concluídos dado um tipo de serviço.

Para este teste o tipo de serviço utilizado foi a reparação, onde foram adicionados três serviços.

Para a fase inicial as datas de início de cada serviço foram iguais sendo cada uma: 22 06 10 00 00 2026 e as datas de conclusão também foram iguais: 23 06 10 00 00 2026.

O que foi alterado entre cada serviço foram os componentes, para o primeiro foi o valor de 100, para o segundo o valor de 50 e para o terceiro o valor de 20. Devido ao funcionamento da aplicação os valores são sempre adicionados ao início.

```
4 - Listar todos os dados
[Codigo Servico ID: 3 Nome do Cliente: Ramiro Observacoes: Ventoinha(20.00) Pago: 0 Codigo Servico ID: 2 Nome do Cliente
: Maria Observacoes: Bateria(50.00) Pago: 0 Codigo Servico ID: 1 Nome do Cliente: Luis Observacoes: RAM(100.00) Pago: 0]
2 - Registrar Tipo de Servico
3 - Apresentar tabela
4 - Editar Dados de um tipo de servico
```

Figura 17 – Dados dos Concluídos antes da ordenação

Após a seleção da opção 16 que é para chamar o subprograma de ordenação os dados ficaram organizados da seguinte maneira:

```
Cliente: Luis
ServicoID: 1
Observacoes: RAM(100.00)
Pago: 0

Data de Conclusao: 23/06/2026 10:00:00

Cliente: Maria
ServicoID: 2
Observacoes: Bateria(50.00)
Pago: 0

Data de Conclusao: 23/06/2026 10:00:00

Cliente: Ramiro
ServicoID: 3
Observacoes: Ventoinha(20.00)
Pago: 0
```

Figura 18 - Ordenação dos dados após chamada da função de ordenação

9.3. Calcular a duração mínima, a duração máxima e a duração média de todos os serviços concluídos

O teste realizado tem como objetivo o cálculo das durações mínimas, máximas e médias de todos os serviços concluídos.

Para a realização deste teste foram adicionados três tipos de serviços diferentes: Reparação, Manutenção e Limpeza.

Para os serviços adicionados a data de início foi a mesma: 22 06 10 00 00 2026.

A opção para proceder aos cálculos foi a opção 18.

Para cada serviço as datas de fim ou de conclusão foram diferentes:

1. Reparação: 24 06 10 00 00 2026
2. Manutenção: 23 06 10 00 00 2026
3. Limpeza: 22 06 11 00 00 2026

```
Maior duracao:  
Codigo Servico: 1  
Nome Cliente: Maria  
Equipamento: PC  
Valor: 172800.00  
Menor Duracao:  
Codigo Servico: 3  
Nome Cliente: Manuel  
Equipamento: PC  
Valor: 3600.00  
Media de duracao: 87600.00
```

Figura 19 - Resultado do cálculo das durações de todos os serviços concluídos

Como é possível observar pela imagem a maior duração corresponde ao serviço de Reparação e a menor duração corresponde ao serviço de Limpeza.

Os valores obtidos estão no formato de **long long** ou seja em segundos.

10. Conclusão

Após a realização dos testes foram alterados dois subprogramas o de carregar os dados e o de ordenar por ordem decrescente os serviços concluídos por valor a pagar.

O subprograma de carregar dados inicial colocava as listas de maneira invertida, ou seja durante a execução da aplicação os serviços são adicionados para o início da lista mas quando o utilizador terminasse de utilizar a aplicação e executasse novamente os dados eram carregados de maneira a que o ultimo serviço ficasse em primeiro.

Com a nova implementação ao carregar os dados os serviços são adicionados a partir do ultimo serviço, assim as listas de serviços ficam na mesma ordem que estavam durante a aplicação.

```
//Carregar Dados
int carregarDados(Servico servicos[]){
    FILE *f = fopen("dados.dat" , "rb");

    if(f == NULL){
        printf("Nao foi possivel abrir o ficheiro\n");
        return 0;
    }
    int totalServicos = 0;

    fread(&totalServicos , sizeof(int) , 1 , f);

    for(int i = 0 ; i<totalServicos ; i++){
        fread(&servicos[i] , sizeof(Servico) , 1 , f);

        servicos[i].execucao = NULL;
        servicos[i].concluidos = NULL;

        servicoIndividual aux;
        pEstrutura pUltimo = NULL;

        fread(&aux , sizeof(servicoIndividual) , 1 , f);

        while(aux.codigoServicoID != -1){

            pEstrutura novo = (pEstrutura)malloc(sizeof(servicoIndividual));

            if(novo != NULL){
                *novo = aux;
                novo -> prox = NULL;

                if(servicos[i].execucao == NULL){
                    servicos[i].execucao = novo;
                }
                else{
                    pUltimo -> prox = novo;
                }

                pUltimo = novo;
            }

            fread(&aux , sizeof(servicoIndividual) , 1 , f);
        }
    }
}
```

Figura 20 - Nova Função de Carregar Dados


```
fread(&aux , sizeof(servicoIndividual) , 1, f);
pUltimo = NULL;

while(aux.codigoServicoID != -1){

    pEstrutura novoConcluidos = (pEstrutura)malloc(sizeof(servicoIndividual));

    if(novoConcluidos != NULL){
        *novoConcluidos = aux;
        novoConcluidos -> prox = NULL;

        if(servicos[i].concluidos == NULL){
            servicos[i].concluidos = novoConcluidos;
        }
        else{
            pUltimo -> prox = novoConcluidos;
        }

        pUltimo = novoConcluidos;
    }

    fread(&aux , sizeof(servicoIndividual) , 1, f);
}

fclose(f);
printf("Servicos carregados com sucesso!\n");
return totalServicos;
}
```

Figura 21 - Nova função de carregar dados

Para concluir este trabalho permitiu consolidar conhecimentos relativos a listas ligadas, algoritmos de ordenação e recursividade.

Com a realização deste trabalho foi possível desenvolver competências a nível de análise de código e a nível de resolução de problemas.

11.Referências

Prompts no Gemini:

“Qual a melhor maneira de guardar os dados em ficheiro binário?”

“Qual seria a melhor função para Aplicar recursividade?”

Vídeos:

[1] CodeVault, “Short Introduction to linked lists in C” , Youtube 16 de Setembro de 2019.

Link: [Short introduction to linked lists in C](#)

Slides dados nas aulas referentes à matéria de listas ligadas.